

ATLAS OS

Product Requirements

Technical & requirements

Program	ATLAS Habitat Initiative
Product	ATLAS OS — the Habitat Twin (a.k.a. Smart admin)
First unit	ATLAS 01
Author	Elijah Royaie
Status	Draft v0.1 — living document
Companion docs	Business Plan · Legal & Compliance · Owner’s Project Requirements · Security & Data Compliance

This is the PRD the README and Business Plan reference. It is the source of truth for **what ATLAS OS must do** and the acceptance criteria each capability is judged against.

1. Purpose & scope

ATLAS OS is the human window into a self sufficient habitat: an interactive **3D digital twin optimization layer**, over live per floor telemetry, incident triage, and resident broadcast. **It is never a dependency** — life safety is held by safety rated building controls (BMS/PLC), not by this app or any cloud service.

In scope (P0): the twin, per floor telemetry, incident feed, broadcast, KPI strip, fleet rollup, sensor ingestion, building health, and resident identity/auth beyond read views, native mobile apps.

Out of scope (P0): direct actuation of life safety systems, financial/billing, resident identity/auth beyond read views, native mobile apps.

2. Personas

Persona	Needs
Operator / building manager	At a glance health, fast incident triage, broadcast to residents
Resident	Trust signals: autonomy %, food/water/energy status, clear incident comms
Investor / donor	A legible, demoable proof the thesis is real (simulator + twin)
Engineer / SRE	Sensor ingestion, data integrity, uptime, auditability

3. Functional requirements

Feature labels mirror the code (F1–F12); each row is testable.

ID	Requirement	Acceptance criteria
----	-------------	---------------------

F1	3D Habitat Twin	Orbitable tower; one floor per need, color coded; floors with an active incident pulse; selecting a floor opens its telemetry
F2	Per floor telemetry	Tapping a floor shows live energy/water/food/occupancy/climate; metrics are sparse tolerant (only what a floor reports)
F3	Security & incident feed	Severity ranked (crit/warn/info), live, "d15 s poll; clicking an event focuses its floor
F4	Resident broadcast	Compose + send; a broadcast mirrors an info incident into the feed; POST validates body, returns 201/400
F5	Building KPI strip	Autonomy %, battery, solar, water reuse, food output, residents, open incidents — derived by aggregating floors
F7	Fleet view	Multi building rollup, separate from the single building console
F11	Sensor ingestion	Tagged sensor points (Brick/Haystack style); ingest + query via /api/sensors; 201/400 contract
F12	Building Simulator	Operating voxel/pixel twin: cut away floors, working elevator(s), switchback stairs, rooftop solar + reservoir, residents; per floor lighting so every level reads
+	Guided walk through	Animated rooftop! 'underground camera descent with annotated callouts
+	WebXR AR	"Enter AR" on WebXR capable devices; graceful fallback to walk through where unsupported

4. Non functional requirements

Area	Requirement
Local first	App renders fully on in memory seed data with no database connected; DATABASE_URL enables optional Neon Postgres persistence
Performance	Twin interactive at 60 fps target on a modern laptop; live model "d 60k tris / "d 2.5 MB; first paint defaults to lightweight (pixel) mode
Responsiveness	Mobile first; verified on Desktop Chrome, Tablet (Galaxy Tab S9), Mobile (Pixel 5) Playwright projects; safe area insets honored

Accessibility	WCAG 2.1 AA targets for DOM chrome; keyboard navigable controls; sufficient contrast in dark ops palette
Internationalization	UI strings translatable; per user preference, multilingual + dark mode variants maintained in parallel
Reliability	Cloud app is optional; degrades to seed/offline; target 99.9% uptime for hosted instances
Security	See Security & Data Compliance; no PII in seed data; least privilege API
Auditability	Incidents/broadcasts logged; sensor ingestion timestamped; CRA/SR&ED compatible R&D logs retained
Privacy by design	Presence sensing is camera free (ESP32 presence nodes); data minimization (see Privacy Policy)

5. Data model (domain types)

[Floor](#), [Incident](#), [Broadcast](#), [BuildingKpis](#), [SensorPoint](#), [Building](#) ([src/lib/types.ts](#)). Enums for colors/labels in [src/lib/ui.ts](#) (kept in sync with [tailwind.config.ts](#)). [FloorMetrics](#) is all optional; KPIs aggregate across floors.

6. APIs

[/api/floors](#), [/api/incidents](#), [/api/broadcasts](#), [/api/sensors](#) — all [force-dynamic](#), [validate](#)

POST bodies, return JSON ([201](#) create / [400](#) bad input). They delegate to the DB optional lib data layer.

7. Architecture constraints

Next.js 14 (App Router) + React 18 + TS strict; three.js via R3F/drei; Tailwind; Drizzle ORM over Neon. The **safety boundary is absolute**: ATLAS OS observes and advises; it does not control fire, egress, electrical isolation, or biogas systems — those are deterministic, [OPR](#) safety rated, and independent of cloud/AI (see [here](#))

8. Testing & release gates

`npm run verify` = typecheck + lint + build + Playwright matrix. CI ([.github/workflows/ci.yml](#)) runs build + e2e on every push to [main](#). A change ships only when these are green.

9. Out of scope / future

Native apps, resident auth & billing, direct actuation, multi tenant org management, on device ML — tracked separately, not P0.

Living document — update alongside the code it specifies.

